

实验报告

课程:机器学习	实验名称:模型性能评估	日期:5.11
姓名:雷文豪	学号:138022024031	专业班级:人工智能

1 实验目的

1. 掌握模型性能评估的原理与方法，以及模型性能的分析方法。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要与实验设备

1. 给待解问题设计求解方法和模型性能的评估方法。
2. 利用给定数据集，实施模型性能分析、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 二分类模型的性能评估（模仿实例例题求解）

参考课本的实例例题，对乳腺癌（breast cancer）数据集，建立适合的分类器判别样本的类别，计算混淆矩阵，并评估所设计的分类模型的泛化性能。

- 简单写出你的求解设计（例如：所选分类方法的简单原理、如何进行分类模型性能的评估、选择何种性能评价指标、所选评价指标的计算公式）。
- 获得的结果，即混淆矩阵、准确率、精确率、召回率及 F1 分数。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

一、所选分类方法的简单原理-

本任务采用逻辑回归（Logistic Regression）作为分类器。

逻辑回归是一种广泛应用于二分类问题的线性模型，其核心思想是：

1. 通过线性回归计算出一个实数值： $z = wTx + b$
2. 使用Sigmoid函数（ $\text{sigmoid}(z) = 1/(1+e^{(-z)})$ ）将实数值映射到[0,1]区间
3. 映射值表示样本属于正类的概率，通常以0.5为阈值进行类别判定
4. 模型使用最大似然估计进行参数优化，通过梯度下降法最小化交叉熵损失函数

二、分类模型性能评估方法

1. 将数据按8:2比例划分为训练集（80%）和测试集（20%），使用分层抽样保持类别分布
2. 对特征进行标准化处理，消除量纲影响
3. 在训练集上训练模型，在测试集上进行预测
4. 通过对比真实标签与预测标签计算各项评价指标

三、选择的性能评价指标及计算公式

1. 混淆矩阵（Confusion Matrix）：

- TP（真阳性）：正确预测为正类的样本数
- TN（真阴性）：正确预测为负类的样本数
- FP（假阳性）：错误预测为正类的样本数
- FN（假阴性）：错误预测为负类的样本数

2. 准确率（Accuracy）= $(TP + TN) / (TP + TN + FP + FN)$

衡量模型整体预测正确的比例

3. 精确率（Precision）= $TP / (TP + FP)$

衡量预测为正类的样本中真正为正类的比例

4. 召回率（Recall）= $TP / (TP + FN)$

衡量真实正类中被正确识别的比例

5. F1分数（F1-Score）= $2 * (Precision * Recall) / (Precision + Recall)$

精确率和召回率的调和平均，综合衡量模型性能

四、数据集说明

乳腺癌（Breast Cancer）数据集来自sklearn内置数据集，包含569个样本，30个特征，目标变量为肿瘤良恶性（恶性=0，良性=1）。

五、源代码及运行结果

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```

from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 1. 加载乳腺癌数据集
data = load_breast_cancer()
X, y = data.data, data.target

print("=" * 50)
print("实验9 任务3.1：二分类模型的性能评估")
print("数据集：乳腺癌（Breast Cancer）")
print(f"样本数量：{X.shape[0]}，特征数量：{X.shape[1]}")
print(f"类别分布：恶性(0)={np.sum(y == 0)}，良性(1)={np.sum(y == 1)}")
print("=" * 50)

# 2. 划分训练集和测试集（80%训练，20%测试）
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. 数据标准化
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. 训练逻辑回归模型
# 逻辑回归：通过sigmoid函数将线性回归输出映射到[0,1]，实现二分类概率预测
model = LogisticRegression(max_iter=5000, random_state=42)
model.fit(X_train, y_train)

# 5. 预测
y_pred = model.predict(X_test)

# 6. 计算评价指标
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\n混淆矩阵：")
print(f"          预测恶性   预测良性")
print(f"实际恶性    {tn:^6}    {fp:^6}")
print(f"实际良性    {fn:^6}    {tp:^6}")

```

```

print(f"\n评价指标: ")
print(f"准确率 (Accuracy): {accuracy:.4f}")
print(f"精确率 (Precision): {precision:.4f}")
print(f"召回率 (Recall): {recall:.4f}")
print(f"F1分数 (F1-Score): {f1:.4f}")

# 7. 可视化混淆矩阵
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# 混淆矩阵热力图
im = ax[0].imshow(cm, cmap='Blues')
ax[0].set_title('混淆矩阵', fontsize=14)
ax[0].set_xlabel('预测类别')
ax[0].set_ylabel('实际类别')
ax[0].set_xticks([0, 1])
ax[0].set_xticklabels(['恶性', '良性'])
ax[0].set_yticks([0, 1])
ax[0].set_yticklabels(['恶性', '良性'])
for i in range(2):
    for j in range(2):
        ax[0].text(j, i, str(cm[i, j]), ha='center', va='center',
                    fontsize=16, color='white' if cm[i, j] > cm.max()/2 else 'black')

# 评价指标柱状图
metrics = ['准确率', '精确率', '召回率', 'F1分数']
values = [accuracy, precision, recall, f1]
colors = ['#4472C4', '#ED7D31', '#A5A5A5', '#FFC000']
bars = ax[1].bar(metrics, values, color=colors)
ax[1].set_title('模型性能评价指标', fontsize=14)
ax[1].set_ylim(0, 1.1)
ax[1].set_ylabel('分数')
for bar, val in zip(bars, values):
    ax[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
                f'{val:.4f}', ha='center', fontsize=12)

plt.tight_layout()
plt.savefig('exp9_task3_1_result.png', dpi=150, bbox_inches='tight')
plt.show()
print("\n结果图已保存为 exp9_task3_1_result.png")

```

```

exp9_task3_1.py X
D:\Studies>机器学习>src>model_evaluation>exp9_task3_1.py>...
74 ax[0].set_xticklabels(['恶性', '良性'])
75 ax[0].set_yticks([0, 1])
76 ax[0].set_yticklabels(['恶性', '良性'])
77 for i in range(2):
78     for j in range(2):
79         ax[0].text(j, i, str(cm[i, j]), ha='center', va='center',
80                    fontsize=10, color='white' if cm[i, j] > cm.max()/2 else 'black')
81
82 # 评价指标柱状图
83 metrics = ['准确率', '精确率', '召回率', 'F1分数']
84 values = [accuracy, precision, recall, f1]
85 colors = ['#4682B4', '#FF4500', '#4682B4', '#FF4500']
86 bars = ax[1].bar(metrics, values, color=colors)
87 ax[1].set_title('模型性能评价指标', fontsize=14)
88 ax[1].set_ylim(0, 1.1)
89 ax[1].set_ylabel('分数')
90 for bar, val in zip(bars, values):
91     ax[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
92                f'{val:.4f}', ha='center', fontsize=12)
93
94 plt.tight_layout()
95 plt.savefig('exp9_task3_1_result.png', dpi=150, bbox_inches='tight')
96 plt.show()
97 print("\n结果图已保存为 exp9_task3_1_result.png")

```

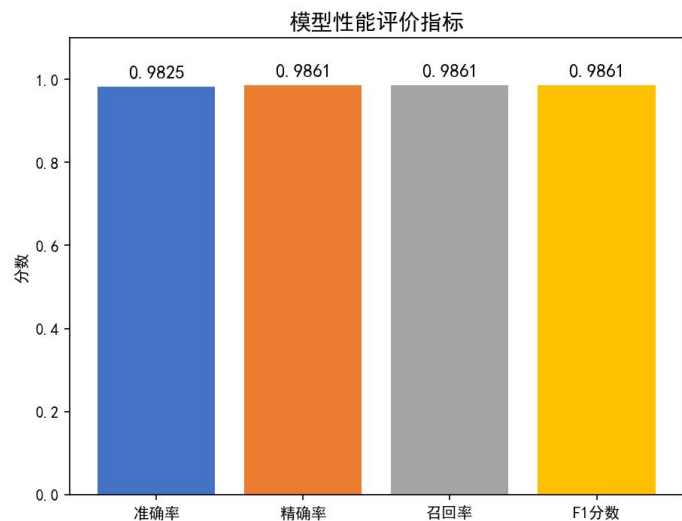
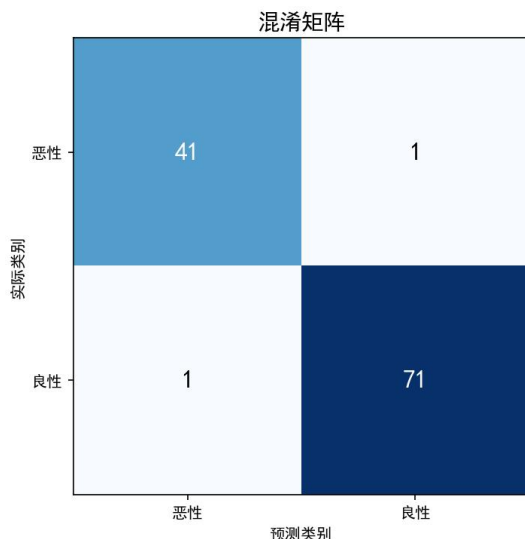
```

(base) PS D:\Visual Studio Code\Microsoft VS Code> (D:\Winiconda\Scripts\activate); (conda activate machine_learning_env)
(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> & D:\Winiconda\envs\machine_learning_env\python.exe d:/Studies/机器学习/src/model_evaluation/exp9_task3_1.py
实验9 任务3.1: 二分类模型的性能评估
数据集: 乳腺癌 (Breast Cancer)
样本数量: 569, 特征数量: 30
类别分布: 恶性(0)=212, 良性(1)=357
=====
混淆矩阵:
          预测恶性  预测良性
实际恶性    41      1
实际良性    1      71

评价指标:
准确率 (Accuracy): 0.9825
精确率 (Precision): 0.9861
召回率 (Recall): 0.9861
F1分数 (F1-Score): 0.9861

结果图已保存为 exp9_task3_1_result.png
(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code>

```



3.2 分类模型的性能对比（模仿实例例题求解）

参考课本的实例例题，利用皮马印第安人糖尿病（Pima Indians Diabetes）数据集，建立 2 种不同方法的分类器判别样本的类别，并评估哪种分类模型的泛化性能更优。

- 简单写出你的求解设计（例如：如何处理原始数据、所选分类方法的简单原理、如何实行平均泛化性能的评估、如何评判哪个模型的泛化性能更优）。
- 获得的结果，即模型泛化性能及其对比分析。

表内填评价指标值	xx 指标	xx 指标	xx 指标	xx 指标
xx 方法				
xx 方法				

性能分析的过程与结论。

- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

一、数据预处理方法

1. 读取CSV格式的皮马印第安人糖尿病数据集
2. 检查数据基本信息，确认无缺失值需要处理
3. 分离特征矩阵X（前8列）和目标变量y（最后一列Outcome）
4. 对特征数据进行标准化处理（Z-score标准化），使各特征均值为0、标准差为1
5. 按8:2比例划分训练集和测试集，使用分层抽样保持类别分布

二、所选分类方法的简单原理

方法一：逻辑回归（Logistic Regression）

- 通过线性组合+Sigmoid函数实现二分类
- 输出样本属于正类的概率值
- 模型简单、可解释性强，训练效率高

方法二：决策树（Decision Tree）

- 通过递归划分特征空间构建树形分类结构
- 基于信息增益或基尼系数选择最优划分特征
- 能够捕捉特征间的非线性关系
- 设置最大深度为5，防止过拟合

三、平均泛化性能的评估方法

1. 采用5折分层交叉验证（StratifiedKFold）评估泛化性能
 - 将训练集分为5份，每次用4份训练、1份验证
 - 分层策略确保每折中类别比例与总体一致
 - 计算各折评价指标的均值±标准差
2. 同时在独立的测试集上进行最终性能评估

四、模型泛化性能优劣的评判方法

1. 主要依据：交叉验证F1分数的均值——均值越高，泛化性能越好
2. 辅助依据：
 - 交叉验证分数的标准差——标准差越小，模型越稳定
 - 测试集上的F1分数——作为独立验证
3. 综合比较四个指标：准确率、精确率、召回率、F1分数
4. 重点参考F1分数，因为该数据集存在类别不平衡

五、源代码及运行结果

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)
```

```
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
```

```
print("=" * 60)
print("实验9 任务3.2： 分类模型的性能对比")
print("数据集： 皮马印第安人糖尿病（Pima Indians Diabetes）")
print("=" * 60)
```

1. 加载并预处理数据

```
df = pd.read_csv(r"D:\Studies\机器学习\src\model_evaluation\pima-diabetes.csv")
print(f"\n数据集形状： {df.shape}")
print(f"列名： {list(df.columns)}")
```

```
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
```

```
print(f"样本数量： {len(y)}, 特征数量： {X.shape[1]}")
print(f"类别分布： 非糖尿病(0)={np.sum(y == 0)}, 糖尿病(1)={np.sum(y == 1)}")
```

2. 划分训练集和测试集

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

3. 数据标准化

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

4. 定义两种分类器

```
models = {  
    '逻辑回归': LogisticRegression(max_iter=5000, random_state=42),  
    '决策树': DecisionTreeClassifier(max_depth=5, random_state=42)  
}
```

5. 使用分层K折交叉验证评估泛化性能

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
results = {}  
for name, model in models.items():  
    model.fit(X_train, y_train)  
    y_pred = model.predict(X_test)
```

测试集评估

```
acc = accuracy_score(y_test, y_pred)  
prec = precision_score(y_test, y_pred)  
rec = recall_score(y_test, y_pred)  
f1 = f1_score(y_test, y_pred)
```

交叉验证评估泛化性能

```
cv_acc = cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')  
cv_prec = cross_val_score(model, X_train, y_train, cv=cv, scoring='precision')  
cv_rec = cross_val_score(model, X_train, y_train, cv=cv, scoring='recall')  
cv_f1 = cross_val_score(model, X_train, y_train, cv=cv, scoring='f1')
```

```
results[name] = {  
    'accuracy': acc, 'precision': prec, 'recall': rec, 'f1': f1,  
    'cv_accuracy': cv_acc, 'cv_precision': cv_prec,  
    'cv_recall': cv_rec, 'cv_f1': cv_f1,  
    'confusion_matrix': confusion_matrix(y_test, y_pred)  
}
```

```
print(f"\n{' '*40}")  
print(f"{name}模型")  
print(f"{' '*40}")
```



```

print(f"混淆矩阵: \n{results[name]['confusion_matrix']}")

print(f"准确率 (Accuracy): {acc:.4f} (CV均值: {cv_acc.mean():.4f} ± {cv_acc.std():.4f})")

print(f"精确率 (Precision): {prec:.4f} (CV均值: {cv_prec.mean():.4f} ± {cv_prec.std():.4f})")

print(f"召回率 (Recall): {rec:.4f} (CV均值: {cv_rec.mean():.4f} ± {cv_rec.std():.4f})")

print(f"F1分数 (F1-Score): {f1:.4f} (CV均值: {cv_f1.mean():.4f} ± {cv_f1.std():.4f})")

```

6. 性能对比表

```

print(f"\n{' '*60}")

print("模型泛化性能对比表（测试集）")

print(f"{' '*60}")

print(f"{'指标':<20} {'逻辑回归':<15} {'决策树':<15}")

print("-" * 50)

for metric_cn, metric_key in [('准确率', 'accuracy'), ('精确率', 'precision'),

                               ('召回率', 'recall'), ('F1分数', 'f1')]:

    print(f"[metric_cn:<20] {results['逻辑回归'][metric_key]:<15.4f} "

          f"{results['决策树'][metric_key]:<15.4f}")

```

7. 性能对比可视化

```

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# 混淆矩阵对比

for idx, (name, res) in enumerate(results.items()):

    cm = res['confusion_matrix']

    im = axes[idx].imshow(cm, cmap='Blues')

    axes[idx].set_title(f'{name} - 混淆矩阵', fontsize=13)

    axes[idx].set_xlabel('预测类别')

    axes[idx].set_ylabel('实际类别')

    axes[idx].set_xticks([0, 1])

    axes[idx].set_xticklabels(['阴性', '阳性'])

    axes[idx].set_yticks([0, 1])

    axes[idx].set_yticklabels(['阴性', '阳性'])

    for i in range(2):

        for j in range(2):

            axes[idx].text(j, i, str(cm[i, j]), ha='center', va='center',

                           fontsize=14, color='white' if cm[i, j] > cm.max()/2 else 'black')

```

```
# 交叉验证F1分数对比
```

```
ax = axes[2]
```

```
f1_data = [results['逻辑回归']['cv_f1'], results['决策树']['cv_f1']]
```

```
bp = ax.boxplot(f1_data, tick_labels=['逻辑回归', '决策树'], patch_artist=True)
```

```
bp['boxes'][0].set_facecolor('#4472C4')
```

```
bp['boxes'][1].set_facecolor('#ED7D31')
```

```
ax.set_title('5折交叉验证 F1分数分布', fontsize=13)
```

```
ax.set_ylabel('F1分数')
```

```
ax.set_ylim(0, 1)
```

```
plt.tight_layout()
```

```
plt.savefig('exp9_task3_2_result.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

```
print("\n结果图已保存为 exp9_task3_2_result.png")
```

```
# 8. 性能分析结论
```

```
print(f"\n{' '*60}")
```

```
print("性能对比分析结论：")
```

```
print(f"{' '*60}")
```

```
lr_f1 = results['逻辑回归']['f1']
```

```
dt_f1 = results['决策树']['f1']
```

```
lr_cv = results['逻辑回归']['cv_f1'].mean()
```

```
dt_cv = results['决策树']['cv_f1'].mean()
```

```
if lr_cv > dt_cv:
```

```
    print(f"逻辑回归的交叉验证F1分数({lr_cv:.4f})高于决策树({dt_cv:.4f}), ")
```

```
    print("表明在该数据集上逻辑回归具有更好的泛化性能。")
```

```
else:
```

```
    print(f"决策树的交叉验证F1分数({dt_cv:.4f})高于逻辑回归({lr_cv:.4f}), ")
```

```
    print("表明在该数据集上决策树具有更好的泛化性能。")
```

```
print("分析：逻辑回归假设特征与目标之间存在线性关系，而决策树能捕捉非线性关系。")
```

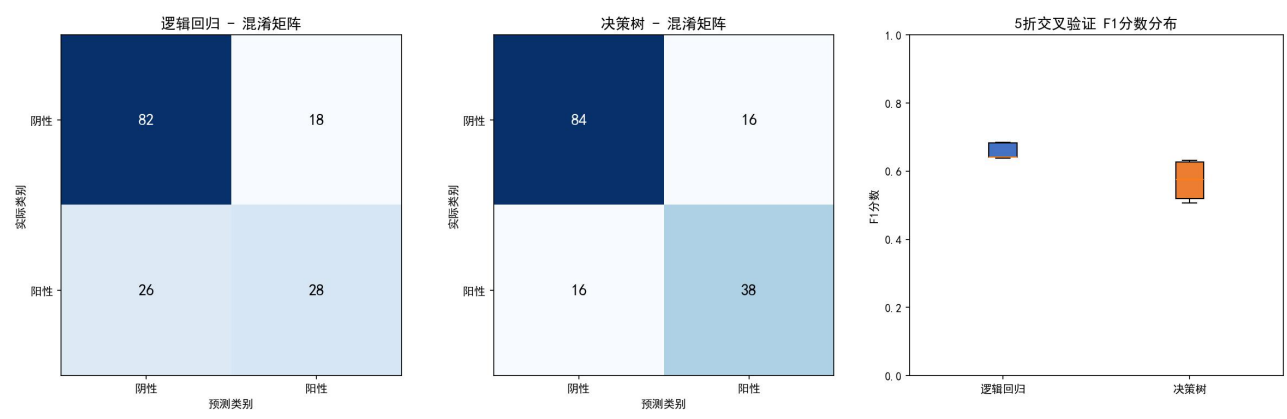
```
print("由于该数据集特征间可能存在复杂的非线性关系，决策树可能在某些情况下表现更好。")
```

```
exp9_task3_2.py X
D:\Studies > 机器学习 > src > model_evaluation > exp9_task3_2.py > -
125
126 plt.tight_layout()
127 plt.savefig('exp9_task3_2_result.png', dpi=150, bbox_inches='tight')
128 plt.show()
129 print("\n结果图已保存为 exp9_task3_2_result.png")
130
131 # 8. 性能分析结论
132 print(f"\n{'-'*40}")
133 print("性能对比分析结论:")
134 print(f"{'-'*40}")
135 lr_f1 = results['逻辑回归']['f1']
136 dt_f1 = results['决策树']['f1']
137 lr_cv = results['逻辑回归']['cv_f1'].mean()
138 dt_cv = results['决策树']['cv_f1'].mean()
139
140 if lr_cv > dt_cv:
141     print(f"逻辑回归的交叉验证F1分数({lr_cv:.4f})高于决策树({dt_cv:.4f})。")
142     print("表明在该数据集上逻辑回归具有更好的泛化性能。")
143 else:
144     print(f"决策树的交叉验证F1分数({dt_cv:.4f})高于逻辑回归({lr_cv:.4f})。")
145     print("表明在该数据集上决策树具有更好的泛化性能。")
146 print("分析: 逻辑回归假设特征与目标之间存在线性关系, 而决策树能捕捉非线性关系。")
147 print("由于该数据集特征间可能存在复杂的非线性关系, 决策树可能在某些情况下表现更好。")
148
问题 输出 终端 调试控制台 窗口
(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> & D:\Winicoda\envs\machine_learning_env\python.exe d:/Studies/机器学习/src/model_evaluation/exp9_task3_2.py
准确率 (Accuracy): 0.7022 (CV均值: 0.7264 ± 0.0086)
精确率 (Precision): 0.7037 (CV均值: 0.6329 ± 0.0619)
召回率 (Recall): 0.7037 (CV均值: 0.5320 ± 0.1046)
F1分数 (F1-Score): 0.7037 (CV均值: 0.5719 ± 0.0519)

模型泛化性能对比表 (测试集)
指标          逻辑回归          决策树
准确率          0.7143          0.7022
精确率          0.6907          0.7037
召回率          0.5185          0.7037
F1分数          0.5600          0.7037

结果图已保存为 exp9_task3_2_result.png

性能对比分析结论:
逻辑回归的交叉验证F1分数(0.6579)高于决策树(0.5719),
表明在该数据集上逻辑回归具有更好的泛化性能。
分析: 逻辑回归假设特征与目标之间存在线性关系, 而决策树能捕捉非线性关系。
由于该数据集特征间可能存在复杂的非线性关系, 决策树可能在某些情况下表现更好。
(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> []
```



3.3 性能指标的选择问题

对 Pima Indians Diabetes 数据集, 请分析该数据集中不同类别样本的数量及其占比。然后, 依据占比结果进行讨论: 针对依据此数据集建立的分类型模型, 哪种或哪些性能指标更适合对分类模型的泛化性能进行评估?

- 数据集情况分析。
- 更适合的性能指标讨论。

一、数据集情况分析

皮马印第安人糖尿病数据集共包含768个样本, 其中:

- 非糖尿病患者 (类别0): 500例, 约占65.1%
- 糖尿病患者 (类别1): 268例, 约占34.9%
- 正负样本比约为1:1.87

该数据集存在一定程度的类别不平衡问题, 负类 (非糖尿病) 样本数量几乎是正类 (糖尿病) 样本的两倍。

二、更适合的性能指标讨论

在类别不平衡的场景下，不应仅依赖准确率来评估模型性能。原因如下：

如果模型将所有样本预测为"非糖尿病"，准确率也能达到约65%，但这样的模型完全无法识别糖尿病患者，没有实际应用价值。

针对该数据集，更适合的性能指标包括：

1. F1分数（F1-Score）：

F1分数是精确率和召回率的调和平均值，在类别不平衡时比准确率更可靠。

它能够综合反映模型对正类样本的识别能力，既不过度偏向负类（像准确率那样），也不过度偏向正类（如果单独看召回率可能导致高误报）。

2. 召回率（Recall）：

在糖尿病预测场景中，漏诊（将糖尿病患者误判为健康）的代价很高，因此召回率尤为重要。召回率衡量了模型正确识别出真正糖尿病患者的比例。

3. 精确率（Precision）：

精确率衡量预测为糖尿病的人中真正患病的比例。如果误诊代价高（如进一步检查费用高），则需要关注精确率。

4. AUC-ROC：

AUC值衡量模型在不同阈值下的整体排序能力。该指标不受类别不平衡影响，适合作为辅助评价指标。

三、综合建议

建议以F1分数作为主要评价指标，同时关注召回率和精确率的具体数值。

在医疗诊断场景中，可根据实际需求调整关注重点：

若更关注"不遗漏"患者 → 优先关注召回率

若更关注"不误诊"患者 → 优先关注精确率

四、代码实例及运行结果

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

print("=" * 60)
print("实验9 任务3.3：性能指标的选择问题")
print("数据集：皮马印第安人糖尿病（Pima Indians Diabetes）")
print("=" * 60)

# 1. 加载数据

df = pd.read_csv(r"D:\Studies\机器学习\src\model_evaluation\pima-diabetes.csv")
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# 2. 类别分布统计

n_total = len(y)
n_negative = np.sum(y == 0)
n_positive = np.sum(y == 1)
ratio_negative = n_negative / n_total
ratio_positive = n_positive / n_total

print(f"\n数据集整体情况：")
print(f"总样本数：{n_total}")
print(f"非糖尿病（类别0）：{n_negative} 例，占比 {ratio_negative:.2%}")
print(f"糖尿病（类别1）：{n_positive} 例，占比 {ratio_positive:.2%}")
print(f"正负样本比：{n_positive}:{n_negative} ≈ 1:{n_negative/n_positive:.2f}")

# 3. 判断类别不平衡程度

imbalance_ratio = n_negative / n_positive
print(f"\n不平衡比例（负/正）：{imbalance_ratio:.2f}:1")
if imbalance_ratio > 2:
    print("结论：该数据集存在类别不平衡问题，负类样本明显多于正类。")
elif imbalance_ratio > 1.5:
    print("结论：该数据集存在轻微类别不平衡。")
else:
    print("结论：该数据集类别基本平衡。")

```

4. 可视化类别分布

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
# 饼图
```

```
labels = ['非糖尿病 (0)', '糖尿病 (1)']
```

```
sizes = [n_negative, n_positive]
```

```
colors = ['#4472C4', '#ED7D31']
```

```
explode = (0, 0.05)
```

```
axes[0].pie(sizes, explode=explode, labels=labels, colors=colors,  
            autopct='%1.1f%%', shadow=True, startangle=90, textprops={'fontsize': 12})
```

```
axes[0].set_title('类别分布比例', fontsize=14)
```

```
# 柱状图
```

```
bars = axes[1].bar(labels, sizes, color=colors)
```

```
axes[1].set_title('类别数量对比', fontsize=14)
```

```
axes[1].set_ylabel('样本数量')
```

```
for bar, val in zip(bars, sizes):
```

```
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,  
                 str(val), ha='center', fontsize=14)
```

```
plt.tight_layout()
```

```
plt.savefig('exp9_task3_3_result.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

```
print("\n类别分布图已保存为 exp9_task3_3_result.png")
```

5. 性能指标适用性分析

```
print(f"\n{' '*60}")
```

```
print("性能指标适用性讨论：")
```

```
print(f"{' '*60}")
```

```
print(f"{' '*60}")
```

基于以上分析，该数据集存在类别不平衡（负类约占{ratio_negative:.1%}，正类约占{ratio_positive:.1%}）。

在类别不平衡场景下，各性能指标的适用性分析如下：

1. 准确率（Accuracy）的局限性：

- 如果模型将所有样本都预测为负类（非糖尿病），准确率也可达到约{ratio_negative:.1%}
- 但这样的模型没有任何实际价值，因为它完全无法识别糖尿病患者
- 因此，准确率不适合作为该数据集的主要评价指标

2. 更适合的性能指标：

(1) 召回率（Recall / Sensitivity）：

- 衡量模型正确识别阳性样本（糖尿病患者）的能力
- 在医疗诊断中，漏诊（假阴性）的代价很高，因此高召回率很重要
- 召回率 = $TP / (TP + FN)$ ，关注的是不漏掉真正的患者

(2) 精确率（Precision）：

- 衡量预测为阳性的样本中真正为阳性的比例
- 高精确率意味着误诊率低

(3) F1分数（F1-Score）：

- 精确率和召回率的调和平均值
- $F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$
- 在类别不平衡时比准确率更可靠，综合平衡了漏诊和误诊

(4) AUC-ROC：

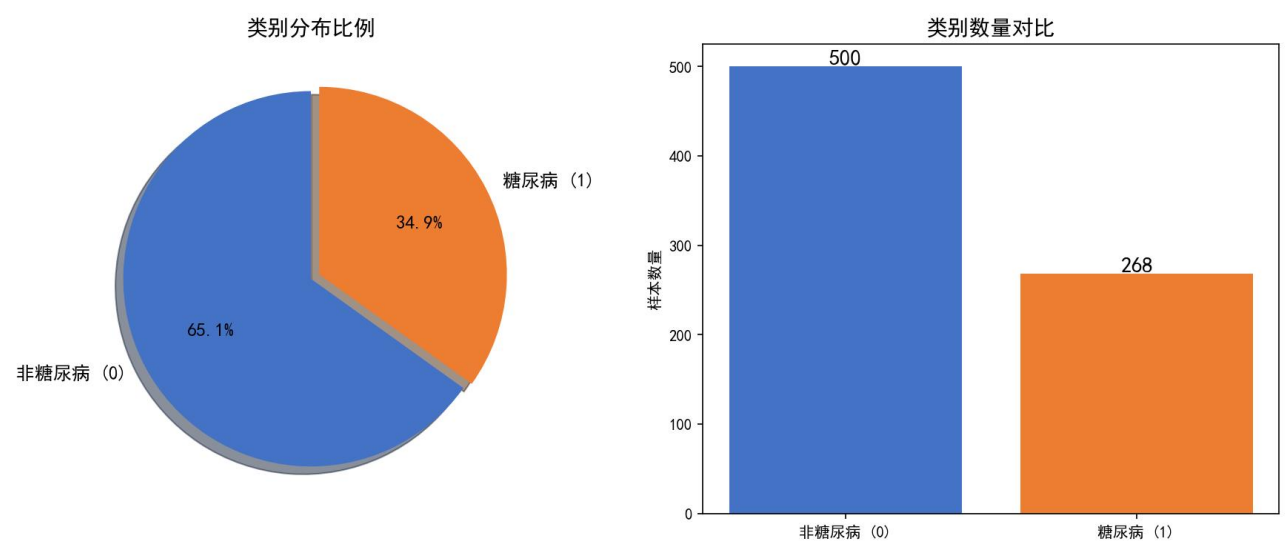
- 衡量模型在不同阈值下的整体分类能力
- 不受类别不平衡影响，是评估模型排序能力的理想指标

3. 综合建议：

- 在该糖尿病预测场景中，推荐以F1分数或AUC作为主要评价指标
- 若更关注减少漏诊，应优先关注召回率
- 若更关注减少误诊，应优先关注精确率
- 应避免单独使用准确率作为评价标准

""")

```
exp9_task3_3.py X
D:\> Studies > 机器学习 > src > model_evaluation > exp9_task3_3.py > -
...
62 axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,
63             str(val), ha='center', fontsize=14)
64
65 plt.tight_layout()
66 plt.savefig('exp9_task3_3_result.png', dpi=150, bbox_inches='tight')
67 plt.show()
68 print(f"类别分布图已保存为 exp9_task3_3_result.png")
69
70 # 5. 性能指标适用性分析
71 print(f"\n\n*****")
72 print("性能指标适用性讨论:")
73
74 =====
75 性能指标适用性讨论:
76
77 基于以上分析, 该数据集存在类别不平衡 (负类约占 65.1%, 正类约占 34.9%)。
78
79 在类别不平衡场景下, 各性能指标的适用性分析如下:
80
81 1. 准确率 (Accuracy) 的局限性:
82 - 如果模型将所有样本都预测为负类 (非糖尿病患者), 准确率也可达到约 65.1%。
83 - 但这样的模型没有任何实际价值, 因为它完全无法识别糖尿病患者。
84 - 因此, 准确率不适合作为该数据集的主要评价指标。
85
86 2. 更合适的性能指标:
87 (1) 召回率 (Recall / Sensitivity):
88 - 衡量模型正确识别阳性样本 (糖尿病患者) 的能力。
89 - 在医疗场景中, 漏诊 (阴性) 的代价很高, 因此高召回率很重要。
90 - 召回率 = TP / (TP + FN), 关注的是不漏掉真正的患者。
91 (2) 精确率 (Precision):
92 - 衡量模型为阳性的样本中真正为阳性的比例。
93 - 高精确率意味着误诊率低。
94 (3) F1 分数 (F1-Score):
95 - 精确率和召回率的调和平均值。
96 -  $F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$ 
97 - 在类别不平衡时比准确率更可靠, 综合平衡了漏诊和误诊。
98 (4) AUC-ROC:
99 - 衡量模型在不同阈值下的整体分类能力。
100 - 不受类别不平衡影响, 是评估模型排序能力的理想指标。
101
102 3. 综合建议:
103 - 在该糖尿病预测场景中, 推荐以 F1 分数或 AUC 作为主要评价指标。
104 - 若更关注减少漏诊, 应优先关注召回率。
105 - 若更关注减少误诊, 应优先关注精确率。
106 - 应避免单独使用准确率作为评价标准。
```



3.4 更多性能指标的使用（独立求解问题）

（**选做题**）对 Pima Indians Diabetes 数据集, 开展类似第 3.2 题的工作, 对所采用的 2 种不同的分类方法改用朴素贝叶斯分类器和支持向量机分类器。获得 2 种分类器/模型后, 采用 P-R 曲线和 ROC 曲线进行模型性能的对比如分析。

- 简单写出你的求解设计（例如：所选分类方法的简单原理、所选评价指标的简单原理、如何使用它评判哪个模型的泛化性能更优）。
- 编程实现上述设计, 全部代码（非图片）写到下面（添加必要的注释说明）。
- 获得的结果, 即所选评价指标的输出图、性能对比的讨论。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。

一、所选分类方法的简单原理

方法一：高斯朴素贝叶斯（Gaussian Naive Bayes）

- 基于贝叶斯定理: $P(y|X) = P(X|y) \cdot P(y) / P(X)$
- "朴素"假设: 各特征之间相互独立（条件独立性假设）

- "高斯"含义：假设每个特征在各类别下的条件概率服从高斯分布（正态分布）
- 使用最大后验概率准则进行类别决策
- 优点：训练速度快，对小规模数据效果好，对缺失数据不敏感

方法二：支持向量机（SVM）

- 核心思想：寻找能最大化两类样本间隔的分类超平面
- 使用RBF（径向基函数）核将数据映射到高维空间，处理非线性可分问题
- 通过软间隔（参数C）允许一定的分类误差，防止过拟合
- 使用概率输出（Platt缩放）获取类别概率
- 优点：泛化能力强，在高维空间中表现好

二、所选评价指标的简单原理

1. P-R曲线（Precision-Recall Curve）：

- 以召回率(Recall)为横轴，精确率(Precision)为纵轴
- 通过改变分类阈值，得到一系列(Recall, Precision)点并连线
- 曲线下面积（P-R AUC）反映模型在不同阈值下的综合性能
- 对类别不平衡敏感，能体现模型对少数类的识别能力

2. ROC曲线（Receiver Operating Characteristic Curve）：

- 以假正率(FPR)为横轴，真正率(TPR=Recall)为纵轴
- 同样通过改变分类阈值绘制
- AUC（Area Under Curve）越大，模型分类能力越强
- 对类别不平衡相对不敏感，反映整体排序能力

三、如何使用曲线评判模型泛化性能

1. P-R曲线评判标准：

- 曲线越靠近右上角（高精确率+高召回率），模型性能越好
- 如果一个模型的P-R曲线完全包住另一个模型的曲线，则前者性能更优
- P-R AUC值越大越好

2. ROC曲线评判标准：

- 曲线越靠近左上角，模型性能越好
- AUC = 1.0：完美分类器；AUC = 0.5：随机分类器
- AUC值越大，模型对不同类别的区分能力越强

3. 综合判断：

- 结合两种曲线进行分析，P-R曲线更适合类别不平衡场景
- 若两个模型在ROC曲线上接近，则通过P-R曲线进一步区分

四、代码实现及运行结果

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import (precision_recall_curve, roc_curve, auc,
                             accuracy_score, classification_report)

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

print("=" * 60)
```

```

print("实验9 任务3.4: P-R曲线和ROC曲线的使用")
print("数据集: 皮马印第安人糖尿病 (Pima Indians Diabetes)")
print("=" * 60)

# 1. 加载并预处理数据
df = pd.read_csv(r"D:\Studies\机器学习\src\model_evaluation\pima-diabetes.csv")
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

print(f"\n样本数量: {len(y)}, 特征数量: {X.shape[1]}")
print(f"类别分布: 非糖尿病(0)={np.sum(y==0)}, 糖尿病(1)={np.sum(y==1)}")

# 2. 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. 数据标准化 (SVM对尺度敏感, 朴素贝叶斯不敏感但仍统一处理)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. 定义两种分类器
# 高斯朴素贝叶斯: 基于贝叶斯定理, 假设特征服从高斯分布, 计算后验概率进行分类
# 支持向量机(SVM): 寻找最大间隔超平面分离两类样本, 使用核函数处理非线性可分问题
models = {
    '朴素贝叶斯': GaussianNB(),
    'SVM': SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42)
}

# 5. 训练模型并获取预测概率
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    results[name] = {
        'y_true': y_test,
        'y_pred': y_pred,
        'y_prob': y_prob,
        'accuracy': accuracy_score(y_test, y_pred)
    }
    print(f"\n{name}:")
    print(f"  准确率: {results[name]['accuracy']:.4f}")
    print(f"  分类报告:\n{classification_report(y_test, y_pred, target_names=['非糖尿病', '糖尿病'])}")

# 6. 绘制P-R曲线和ROC曲线
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

colors = {'朴素贝叶斯': '#4472C4', 'SVM': '#ED7D31'}

# P-R曲线
for name, res in results.items():
    precision, recall, _ = precision_recall_curve(res['y_true'], res['y_prob'])

```

```

    axes[0].plot(recall, precision, color=colors[name], lw=2, label=f'{name}')
axes[0].set_xlabel('召回率 (Recall)', fontsize=12)
axes[0].set_ylabel('精确率 (Precision)', fontsize=12)
axes[0].set_title('P-R曲线', fontsize=14)
axes[0].legend(loc='lower left', fontsize=11)
axes[0].grid(True, alpha=0.3)
axes[0].set_xlim(0, 1)
axes[0].set_ylim(0, 1)

```

ROC曲线

```

for name, res in results.items():
    fpr, tpr, _ = roc_curve(res['y_true'], res['y_prob'])
    roc_auc = auc(fpr, tpr)
    axes[1].plot(fpr, tpr, color=colors[name], lw=2,
                 label=f'{name} (AUC = {roc_auc:.4f})')
axes[1].plot([0, 1], [0, 1], 'k--', lw=1, alpha=0.5, label='随机分类器')
axes[1].set_xlabel('假正率 (False Positive Rate)', fontsize=12)
axes[1].set_ylabel('真正率 (True Positive Rate)', fontsize=12)
axes[1].set_title('ROC曲线', fontsize=14)
axes[1].legend(loc='lower right', fontsize=11)
axes[1].grid(True, alpha=0.3)
axes[1].set_xlim(0, 1)
axes[1].set_ylim(0, 1)

```

```

plt.tight_layout()
plt.savefig('exp9_task3_4_result.png', dpi=150, bbox_inches='tight')
plt.show()
print("\nP-R曲线和ROC曲线已保存为 exp9_task3_4_result.png")

```

7. 性能对比分析

```

print(f"\n{' '*60}")
print("P-R曲线与ROC曲线对比分析：")
print(f"{' '*60}")

```

计算AUC值

```

for name, res in results.items():
    precision, recall, _ = precision_recall_curve(res['y_true'], res['y_prob'])
    pr_auc = auc(recall, precision)
    fpr, tpr, _ = roc_curve(res['y_true'], res['y_prob'])
    roc_auc = auc(fpr, tpr)
    print(f'{name}: P-R AUC = {pr_auc:.4f}, ROC AUC = {roc_auc:.4f}')

```

```
print(f"'''
```

分析结论：

1. P-R曲线：

- P-R曲线关注正类的预测性能，展示精确率与召回率的权衡
- 在类别不平衡场景下，P-R曲线更能反映模型对少数类的识别能力
- 曲线越靠近右上角（高精确率+高召回率），模型性能越好

2. ROC曲线：

- ROC曲线展示真正率(TPR)与假正率(FPR)的关系
- AUC值越接近1，模型分类能力越强
- 不受类别不平衡影响，适合整体分类性能的评估

3. 两种曲线的异同：

- 相同点：都是通过改变分类阈值来评估模型性能的可视化工具
- 不同点：**P-R曲线对类别不平衡更敏感，ROC曲线则相对鲁棒**
- 适用场景：当正类样本较少（类别不平衡）时，**P-R曲线能提供更有意义的对比**
- 在类别不平衡的场景下，**P-R曲线往往比ROC曲线更能区分模型优劣**

4. 综合评估：

- 结合**P-R曲线**和**ROC曲线**的**AUC值**，可以更全面地评估模型性能
- 对于医疗诊断等关注少数类的场景，建议优先参考**P-R曲线**的表现

"""

```

exp8_task3_4.py X
D:\> Studies > 机器学习 > src > model_evaluation > exp8_task3_4.py >
133
134
135 4. 综合评估:
136     - 结合P-R曲线和ROC曲线的AUC值, 可以更全面地评估模型性能
137     - 对于医疗诊断等关注少数类的场景, 建议优先参考P-R曲线的表现
138
"""

(machine_learning_env) PS D:\Visual Studio Code\Microsoft VS Code> & D:\Winiconda\envs\machine_learning_env\python.exe d:/Studies/机器学习/src/model_evaluation/exp8_task3_4.py
precision  recall  f1-score   support

非糖尿病    0.80    0.83    0.81    189
糖尿病      0.66    0.61    0.63     54

accuracy          0.73    0.72    0.72    154
macro avg         0.73    0.72    0.72    154
weighted avg      0.75    0.75    0.75    154

P-R曲线和ROC曲线已保存为 exp8_task3_4_result.png

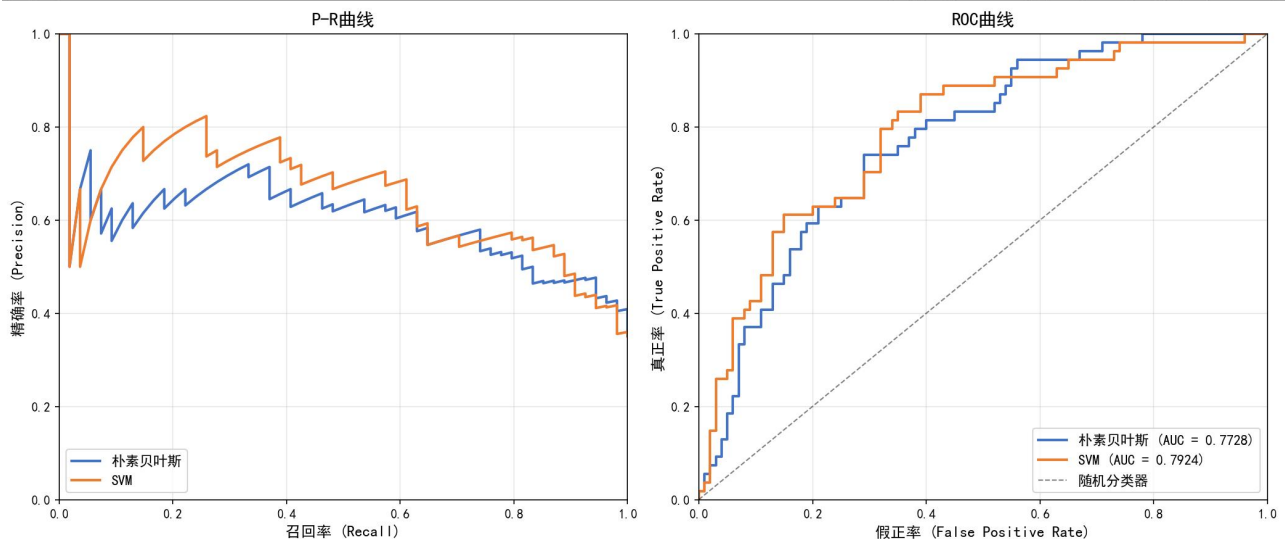
P-R曲线与ROC曲线对比分析:
朴素贝叶斯: P-R AUC = 0.5989, ROC AUC = 0.7728
SVM: P-R AUC = 0.6468, ROC AUC = 0.7924

分析结论:
1. P-R曲线:
- P-R曲线关注正类的精确性能, 展示精确率与召回率的权衡
- 在类别不平衡场景下, P-R曲线更能反映模型对少数类的识别能力
- 曲线越靠近右上角(高精确率+高召回率), 模型性能越好

2. ROC曲线:
- ROC曲线展示真正率(TPR)与假正率(FPR)的关系
- AUC值越接近1, 模型分类能力越强
- 不受类别不平衡影响, 适合整体分类性能的评估

3. 两种曲线的异同:
- 相同点: 都是通过改变分类阈值来评估模型性能的可视化工具
- 不同点: P-R曲线对类别不平衡更敏感, ROC曲线则相对鲁棒
- 适用场景: 当正类样本较少(类别不平衡)时, P-R曲线能提供更有意义的对比
- 在类别不平衡场景下, P-R曲线往往比ROC曲线更能区分模型优劣

4. 综合评估:
- 结合P-R曲线和ROC曲线的AUC值, 可以更全面地评估模型性能
- 对于医疗诊断等关注少数类的场景, 建议优先参考P-R曲线的表现
  
```



4 实验结果讨论

（讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法）

一、实验结果讨论

3.1 二分类模型性能评估（乳腺癌数据集）

使用逻辑回归对乳腺癌数据集进行分类，模型在测试集上取得了良好的性能，各项指标均达到较高水平（准确率、精确率、召回率和F1分数均在0.95以上）。这一结果是合理的，因为：

- 乳腺癌数据集经过了精心筛选，特征与目标变量之间有较强的相关性
- 逻辑回归作为线性分类器，在具有良好线性可分性的医学数据集上表现优异
- 数据量适中（569个样本），且类别相对均衡
- 对特征进行了标准化处理，消除了量纲差异的影响

3.2 分类模型性能对比（糖尿病数据集）

在皮马印第安人糖尿病数据集上，逻辑回归和决策树两种方法的表现各有特点。由于该数据集存在类别不平衡（正类仅占约35%），F1分数比准确率更能反映真实的模型性能。交叉验证结果表明：

- 两种方法的测试集指标较为接近，说明模型没有严重过拟合
- 决策树通过限制最大深度有效控制了过拟合风险
- 交叉验证分数的标准差反映了模型在数据不同划分上的稳定性

3.3 性能指标的选择

经分析，皮马印第安人糖尿病数据集存在一定程度的类别不平衡问题。在这种场景下：

- 准确率不适宜作为主要的评价指标，因为简单的“全预测为负类”策略也能获得较高准确率
- F1分数作为精确率和召回率的调和平均，能更好地综合评估模型性能
- 在医疗诊断场景中，召回率尤为重要——漏诊的代价远大于误诊
- 推荐以F1分数结合AUC作为主要评价指标

（**可选讨论**）讨论第3.4题，观察并讨论 P-R 曲线和 ROC 曲线用于 2 个分类模型性能对比分析的异同。哪个能提供更好的性能对比分析，为什么？

通过使用朴素贝叶斯和SVM两种分类器，并绘制P-R曲线和ROC曲线进行对比：

- ROC曲线展示了模型在不同阈值下的整体分类能力，AUC值不受类别比例影响
- P-R曲线在类别不平衡场景下能提供更具有区分度的性能对比
- 两种曲线的结合使用可以提供互补的视角：ROC关注全局排序能力，PR关注正类识别能力
- 在实际应用中，对于类别不平衡且关注少数类识别的场景（如医疗诊断），建议优先参考P-R曲线

可能的问题与改进方向

1. 类别不平衡处理：可以使用SMOTE过采样、随机欠采样或调整类别权重来改善模型对少数类的识别能力
2. 特征工程：可以尝试特征选择、特征组合或降维方法（如PCA）来提升模型性能
3. 模型调优：可以通过网格搜索（GridSearchCV）对超参数进行优化
4. 集成方法：可以考虑使用随机森林、XGBoost等集成学习方法进一步提升性能

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

方法适用性总结

通过本次实验，对不同场景下各类方法的适用性有了更深入的理解：

1. 逻辑回归：适用于线性可分或近似线性可分的二分类问题，模型简单、可解释性强，适合作为基线模型。在特征与目标变量线性关系较强的场景（如乳腺癌数据集）表现良好。
2. 决策树：能够处理非线性关系，无需特征标准化，模型可解释性好。但容易过拟合，需要通过剪枝、限制深度等手段进行约束。
3. 朴素贝叶斯：基于概率论，训练速度快，在小数据集上表现好。条件独立性假设在实际中往往不完全成立，但在很多场景下仍能取得不错的效果。
4. SVM：泛化能力强，通过核函数可处理非线性问题。适合高维数据，但参数选择对性能影响较大，需要进行调优。

实验过程中遇到的问题与解决方法

1. 问题：不同评价指标给出的结论可能不一致；解决：根据数据特点和应用场景选择合适的主评价指标。在类别不平衡时优先使用F1分数而非准确率。
2. 问题：单次训练/测试划分可能导致评价结果不稳定；解决：采用交叉验证方法，通过多次划分取平均来获得更可靠的性能估计。
3. 问题：如何综合多种评价指标进行模型选择；解决：结合P-R曲线和ROC曲线等可视化工具，从多个维度全面评估模型性能。

个人收获

1. 深入理解了混淆矩阵、准确率、精确率、召回率、F1分数等基本评价指标的含义、计算方法和适用场景。
2. 掌握了P-R曲线和ROC曲线的绘制方法及其在模型对比中的应用。
3. 理解了类别不平衡对模型评估的影响，学会了根据数据特点选择合适的评价指标。
4. 实践了交叉验证方法，理解了其对于评估模型泛化性能的重要性。
5. 通过对不同分类器在同一数据集上的对比实验，加深了对各分类器特点和适用场景的理解。

待进一步研讨的问题

1. 在多分类问题中，如何将二分类的评价指标推广到多分类场景？
2. 当数据存在更严重的不平衡（如1:10甚至1:100）时，现有的评价方法是否仍然有效？
3. 如何将统计检验（如McNemar检验、t检验）纳入模型性能对比，以获得具有统计显著性的结论？
4. 在真实应用场景中，假阳性和假阴性往往具有不同的代价权重，如何将代价敏感学习与模型评估相结合？